

TradeForge — Implementation Roadmap

Total estimated timeline: **16–18 weeks** for a solo/small-team build to a paper-trading-validated MVP, before public launch with real funds.

Two decisions must be locked in before Phase 1 starts, or later phases will build on shaky ground:

Bot config ownership — Python is the only writer of anything bot-related (lifecycle *and* parameters).

ASP.NET only reads for display.

Live data path — frontend connects directly to Python for the live signal/bot-status stream (recommended), or ASP.NET becomes an explicit relay (Redis pub/sub or gRPC streaming). Pick one.

Phase 1 — Foundations (Weeks 1-2)

Component	Details
Market data provider	Select and integrate (broker API, OANDA, Polygon, Twelve Data, or MetaTrader terminal feed). This unblocks every later phase — do it first.
PostgreSQL schema v1	<code>users</code> , <code>strategies</code> , <code>user_bots</code> , <code>trades</code> , <code>signals</code> tables; document table ownership (which service writes to which table).
Redis setup	Session storage, rate limiting, cache.
ASP.NET Core skeleton	Project scaffold, JWT auth (login/register/refresh), user model.
Python service skeleton	FastAPI project scaffold, DB connection, basic health-check endpoint.
Dev/paper trading account	Set up a demo broker account to test against for the rest of the build — never touch a live account until Phase 5 is validated.

Phase 2 — Trading Core (Weeks 3–5)

Component	Details
Indicator library	RSI, MACD, moving averages, Bollinger Bands, etc. (pandas/numpy/TA-Lib).
Strategy definition format	JSONB schema for entry/exit/risk rules; a few hardcoded reference strategies.
Signal generation engine	Evaluates live/historical data against strategy rules, produces signals with confidence scores.
Backtesting engine	Runs a strategy against historical data, computes P&L, win rate, drawdown.
Paper trading mode	Simulates order fills without touching a real broker — this is what you validate everything else against.
Unit + validation tests	Confirm indicator math and backtest results against known reference values before building UI on top.

Phase 3 — Service Integration (Weeks 5–6)

Component	Details
ASP.NET ↔ Python contract	Finalize REST vs gRPC (recommend: gRPC streaming for live signal/status feed, REST for CRUD like start/stop/backtest).
Thin gateway endpoints	Auth, billing, strategy metadata (read-only), bot config (read-only), proxy calls to Python.
Live data path	Implement the decision from the top of this doc — direct frontend↔Python WebSocket/gRPC stream, or Redis pub/sub relay through ASP.NET.
Strategies API	<code>/api/strategies</code> , <code>/api/strategies/{id}</code> — served from hardcoded/seeded data initially.

Phase 4 — Frontend (Weeks 6–9)

Component	Details
Dashboard	Account overview, active bots, recent signals.
Strategies Library UI	Strategy cards, entry/exit rules, "Apply to Chart" / "Deploy Bot" actions.
Chart page	TradingView Lightweight Charts, live price feed, indicators, drawing tools.
Backtest UI	Trigger a backtest, display P&L results and equity curve.
Bot management UI	List bots, status, start/stop/pause controls, parameter editing.
Auth flows	Login, register, password reset, session handling.
Billing UI	Subscription tier display, upgrade/downgrade flow (Stripe Checkout/Portal).

Phase 5 — Broker Integration (Weeks 9–12) — highest-risk phase, budget extra time here

Component	Details
MetaTrader bridge	ZeroMQ/socket bridge between MT4/MT5 EA and Python, or MetaApi SDK integration — pick one path early.
Order placement	Idempotent order submission (safe retries, no duplicate fills).
State reconciliation	On bot restart or reconnect, compare Python's believed state against the broker's actual open positions/orders and resolve mismatches.
Slippage/partial fill handling	Correctly record partial fills and price deviation from expected entry.
Reconnect/failure handling	Bridge disconnects should not silently corrupt bot state; consider isolating the broker bridge from signal generation so one failure doesn't take down the other.
End-to-end paper trading validation	Full loop — signal → order → fill → reconciliation — tested repeatedly against the demo account before Phase 6.

Phase 6 — Bot Deployment System (Weeks 12–14)

Component	Details
Bot lifecycle service	Start/stop/pause, persisted state, owned entirely by Python.
Parameter editing	Live parameter changes (lot size, risk %, stop-loss) without redeploying.
Monitoring & alerting	Error detection, notification on bot failure or broker disconnect (email/push).
Admin visibility	Internal view of all users' bot health — needed before you have real users to support.
Extended paper-mode soak test	Run bots unattended in paper mode for several days to catch state-drift or memory issues before allowing real funds.

Phase 7 — Learn/Guide & Content (Weeks 14–16)

Component	Details
Course structure	Trading fundamentals, technical analysis, indicators/strategies, risk management, trading psychology.
Content authoring	Written modules per the 5-module structure from the original blueprint; video optional.
Glossary	Searchable term reference.
Onboarding flow	First-time user walkthrough tying dashboard, strategies, and charts together.

Phase 8 — Compliance & Monetization (Weeks 16–18)

Component	Details
Legal review	Terms of Service, risk disclaimers, and a check on whether "signals + bots + subscriptions" triggers financial-services regulation in your jurisdiction (FSCA, SEC, FCA, ASIC, etc. depending on where you operate/market). Do this before public launch, not after.
Stripe integration	Subscription tiers, webhooks, upgrade/downgrade/cancel handling.
Feature gating	Tier-based access to strategies, bot count limits, etc.
Downloadable EA packaging	.ex4/.ex5 build, install guide with screenshots, config docs — if you're keeping the downloadable-bot option alongside the cloud bot.

Cross-cutting (ongoing throughout, not a discrete phase)

Component	When to start
Logging & monitoring (app + infra)	From Phase 1 — retrofit is painful once bots are live.
Error tracking (e.g. Sentry)	From Phase 1.
CI/CD pipeline	From Phase 1-2.
Table/service ownership documentation	Before Phase 3, keep updated.
Security review of credential storage (broker API keys/tokens)	Before Phase 5 — this is your biggest liability surface.